RASIKA KHANTE CS23172 PRACTICAL NO 6=To design and implement an Artificial Neural Network for classification problem and evaluate its learning performance.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
path= "/content/drive/MyDrive/ml data set/churn.csv"
df=pd.read_csv(path)
df.head(5)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMemb
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	

```
df.shape
```

```
(10000, 14)
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df.columns
```

```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
       'IsActiveMember', 'EstimatedSalary', 'Exited'],
      dtype='object')
```

```
df.isnull().sum()
```

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0
Exited	0

dtype: int64

```
df[df.isnull().any(axis = 1)]
```

RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember

```
df.describe
```

pandas.core.generic.NDFrame.describe
def describe(percentiles=None, include=None, exclude=None) -> Self

Generate descriptive statistics.

Descriptive statistics include those that summarize the central
tendency, dispersion and shape of a
dataset's distribution, excluding ``NaN`` values.

```
#X = pd.get_dummies(X)
```

```
X = pd.get_dummies(X, dtype = int)
```

```
X.head()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	...	Su
0	1	15634602	619	42	2	0.00	1	1	1	101348.88	...	
1	2	15647311	608	41	1	83807.86	1	0	1	112542.58	...	
2	3	15619304	502	42	8	159660.80	3	1	0	113931.57	...	
3	4	15701354	699	39	1	0.00	2	0	0	93826.63	...	
4	5	15737888	850	43	2	125510.82	1	1	1	79084.10	...	

5 rows × 2947 columns

```
X.head()
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	...	Su
0	1	15634602	619	42	2	0.00	1	1	1	101348.88	...	
1	2	15647311	608	41	1	83807.86	1	0	1	112542.58	...	
2	3	15619304	502	42	8	159660.80	3	1	0	113931.57	...	
3	4	15701354	699	39	1	0.00	2	0	0	93826.63	...	
4	5	15737888	850	43	2	125510.82	1	1	1	79084.10	...	

5 rows × 2947 columns

```
X.shape
```

```
(10000, 2947)
```

```
X.columns
```

```
Index(['RowNumber', 'CustomerId', 'CreditScore', 'Age', 'Tenure', 'Balance',
       'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary',
       ...
       'Surname_Zubarev', 'Surname_Zubareva', 'Surname_Zuev', 'Surname_Zuyev',
       'Surname_Zuyeva', 'Geography_France', 'Geography_Germany',
       'Geography_Spain', 'Gender_Female', 'Gender_Male'],
      dtype='object', length=2947)
```

```
X = df.iloc[:,3:-1]
print(X)
```

```
X head after one-hot encoding:
   CreditScore  Age  Tenure    Balance  NumOfProducts  HasCrCard  \
0          619   42       2       0.00              1          1
1          608   41       1   83807.86              1          0
2          502   42       8  159660.80              3          1
3          699   39       1       0.00              2          0
4          850   43       2  125510.82              1          1

   IsActiveMember  EstimatedSalary  Geography_France  Geography_Germany  \
0               1        101348.88                 1                  0
1               1        112542.58                 0                  0
2               0        113931.57                 1                  0
3               0         93826.63                 1                  0
4               1         79084.10                 0                  0

   Geography_Spain  Gender_Female  Gender_Male
0                0              1            0
1                1              1            0
2                0              1            0
3                0              1            0
4                1              1            0

y head:
0    1
1    0
2    1
3    0
4    0
Name: Exited, dtype: int64
```

```
y = df.iloc[:,-1]
print(y)
```

```
0       1
1       0
2       1
3       0
4       0
       ..
9995    0
9996    0
9997    1
9998    1
9999    0
Name: Exited, Length: 10000, dtype: int64
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.7,test_size=0.25, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
X_train
```

```
array([[ 1.39856221,  1.21282685, -0.34459497, ...,  1.73073215,
         1.09823226, -1.09823226],
       [-0.84292909, -1.49858377, -0.09518109, ..., -0.57779016,
         1.09823226, -1.09823226],
       [ 0.93716027,  1.09130779, -0.94734518, ..., -0.57779016,
        -0.91055421,  0.91055421],
       ...,
       [ 0.12320704, -1.62686706,  0.86090545, ..., -0.57779016,
         1.09823226, -1.09823226],
       [-1.44715417,  1.63407805,  0.15423279, ..., -0.57779016,
        -0.91055421,  0.91055421],
       [ 0.77492428,  0.21993537,  0.46600014, ..., -0.57779016,
        -0.91055421,  0.91055421]])
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

```
ann=tf.keras.models.Sequential()
```

```
ann.add(tf.keras.layers.Dense(units=6,activation="relu"))
```

```
ann.add(tf.keras.layers.Dense(units=1,activation="sigmoid"))
```

```
ann.compile(optimizer="adam",loss="binary_crossentropy",metrics=['accuracy'])
```

```
ann.fit(X_train,y_train,batch_size=32,epochs =100)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-4001886870.py in <cell line: 0>()
----> 1 ann.fit(X_train,y_train,batch_size=32,epochs =100)
```